nearest neighbor lie on opposite sites of the split line for `D`.
So, once we reach the end of the path, we need to backtrack to check other branches.

But even that is not enough: after finding the region containing `P`, we can't be sure that in neighboring regions there isn't one or more points even closer than the closest point we have found during this first traversal.

Figure 9.21 exemplifies this situation perfectly: so far, we have found that `D` is the closest point (among those visited) to `P`, so the real nearest neighbor can't be at a distance larger than the one between `D` and `P`: we can thus trace a circle (a hyper-sphere, in higher dimensions) centered at `P`, and with radius equal to `dist(D, P)`. If this circle intersects other partitions, then those regions might contain a point closer than `D`, and we need to get to them.

How do we know if a region intersects our current nearest neighbor's hyper-sphere? That's simple: each region stems by the partitioning created by split lines; when traversing a path, we go on one side of the split (the one on the same side than `P`), but if the distance between a split line and `P` is less than the distance to our current nearest neighbor, then the hyper-sphere intersects the other partition as well.

To make sure we visit all partitions still intersecting the NN hyper-sphere, we need to back-track our traversal of the tree; in our example, we go back to node `D`, then check the distance between `P` and the vertical split line passing through `D` (which, in turn, is just the difference of the `x`